Security Audit Report

Gno Sprint 3

v1.0

March 25, 2025

Table of Contents

Table of Contents 2
License 3
Disclaimer 4
Introduction 5
Purpose of This Report 5
Codebase Submitted for the Audit 5
Methodology 7
Functionality Overview 7
How to Read This Report 8
Code Quality Criteria 9
Summary of Findings 10
Detailed Findings 11
1. Overlapping account flags may cause conflicting permissions 11
2. Missing key validation allows overwriting internal VM state 11
3. Storing UnrestrictedAddrs in parameters leads to misleading configuration 12
4. Missing parameter validation allows malformed parameter persistence 12
5. Missing struct type validation risks runtime panics during parameter processing 13
6. Unrestricted access to coin's lock and unlock proposal creation functions 13
7. Missing validation in willSetKeeperParams 14
8. Incomplete genesis export 14
9. Partial genesis validation 15
10. Non-idiomatic bitwise operations 15
11. Missing explicit realm path check reduces code clarity 16
12. Missing parameter getters limit runtime validation and debugging 16
13. Usage of panics instead of structured error handling 17
14. Redundant validations 17
15. Missing check for distinct sender and recipient allows redundant transfers 18
16. Outstanding TODO and XXX comments 18
17. Misleading comments 19

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by NewTendermint LLC to perform a security audit of Gno.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/gnolang/gno
Scope	The scope is restricted to the changes applied in the https://github.com/gnolang/gno/pull/3860 pull request reviewed at commit `d24efa034173cb57168e27afdb3dcce4a07767fb`, base branch at `a5e084cc668cbd55eb324f2c8970b8e0aca2b584` in the following paths: • `examples/gno.land/r/sys/params` • `gno.land/pkg/gnoland` • `gno.land/pkg/sdk/vm`

	• `gnovm/stdlibs/sys/params` • `tm2/pkg/sdk/auth` • `tm2/pkg/sdk/bank` • `tm2/pkg/params` • `tm2/pkg/std`

Repository	https://github.com/gnolang/gno
Commit	`d24efa034173cb57168e27afdb3dcce4a07767fb`
Scope	The audit scope is restricted to the `tm2/pkg/sdk/bank` module.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Gno is an interpreted and fully-deterministic implementation of the Go programming language, designed to build succinct and composable smart contracts.

The first blockchain to use it is gno.land, a Proof of Contribution-based chain, backed by a variation of the Tendermint consensus engine.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**High**	The codebase in scope includes the interaction of multiple components, including the gno.land chain, the GnoVM, and Tendermint. The chain incorporates unique concepts and while smart contracts are written in Gno, a new language derived from Go.
Code readability and clarity	**Medium**	The codebase includes numerous development leftovers such as commented-out code.
Level of documentation	**Medium-High**	The provided documentation is extensive.
Test coverage	**Medium-High**	Test coverage reported by `go test` for `gno.land/pkg` is `69.80%`. Test coverage reported by `go test` for `tm2/pkg` is `65.60%`. Gno smart contracts do not provide tooling for computing test coverage.

Summary of Findings

No	Description	Severity	Status
1	Overlapping account flags may cause conflicting permissions	Minor	Reported
2	Missing key validation allows overwriting internal VM state	Minor	Reported
3	Storing `UnrestrictedAddrs` in parameters leads to misleading configuration	Minor	Reported
4	Missing parameter validation allows malformed parameter persistence	Minor	Reported
5	Missing struct type validation risks runtime panics during parameter processing	Minor	Reported
6	Unrestricted access to coin's lock and unlock proposal creation functions	Minor	Reported
7	Missing validation in `willSetKeeperParams`	Minor	Reported
8	Incomplete genesis export	Minor	Reported
9	Partial genesis validation	Minor	Reported
10	Non-idiomatic bitwise operations	Informational	Reported
11	Missing explicit realm path check reduces code clarity	Informational	Reported
12	Missing parameter getters limit runtime validation and debugging	Informational	Reported
13	Usage of panics instead of structured error handling	Informational	Reported
14	Redundant validations	Informational	Reported
15	Missing check for distinct sender and recipient allows redundant transfers	Informational	Reported
16	Outstanding `TODO` and `XXX` comments	Informational	Reported
17	Misleading comments	Informational	Reported

Detailed Findings

1. Overlapping account flags may cause conflicting permissions

Severity: Minor

In `gno.land/pkg/gnoland/types.go:52`, the `validFlags` variable is defined using a bitwise `OR` to combine `flagRealmAccount` and `flagValidatorAccount`, which are intended to be mutually exclusive.

However, this implementation permits an account to be simultaneously flagged as both a validator and a realm account. Such a state violates expected permission boundaries and could lead to incorrect privilege assignments or misinterpretation of account roles.

Recommendation

We recommend enforcing mutual exclusivity by updating flag validation to ensure only one of `flagRealmAccount` or `flagValidatorAccount` can be set at a time.

Status: Reported

This audit was performed on an early version of the protocol.

2. Missing key validation allows overwriting internal VM state

Severity: Minor

In `gno.land/pkg/sdk/vm/genesis.go:56-58`, the code sets VM state keys using the pattern `"vm:" + rp.Key` for each entry in `gs.RealmParams`.

However, the value of `rp.Key` is not validated to exclude reserved or sensitive keys such as `"p"`, which is internally used by the VM keeper. If an external realm parameter uses a reserved key like `"p"`, it could overwrite critical internal state values during genesis initialization.

This flaw introduces a risk of state corruption or unexpected behavior during VM execution.

Recommendation

We recommend adding validation to ensure `rp.Key` cannot equal reserved values like `"p"`.

Status: Reported

This audit was performed on an early version of the protocol.

3. Storing `UnrestrictedAddrs` in parameters leads to misleading configuration

Severity: Minor

In `tm2/pkg/sdk/auth/params.go:37`, the `UnrestrictedAddrs` field is defined within the parameter set but is only referenced during genesis initialization. This design choice introduces ambiguity, as the presence of `UnrestrictedAddrs` in parameters implies it can be updated at runtime to dynamically affect account restriction behavior.

However, since the field is not consulted beyond genesis, any updates made post-initialization have no functional effect, potentially misleading developers into assuming support for dynamic permission changes.

Recommendation

We recommend removing `UnrestrictedAddrs` from parameters and handling it exclusively during genesis initialization. If dynamic updates are required, implement validation or state migration logic to ensure changes to `UnrestrictedAddrs` trigger proper account restriction updates.

Status: Reported

This audit was performed on an early version of the protocol.

4. Missing parameter validation allows malformed parameter persistence

Severity: Minor

In `tm2/pkg/sdk/auth/params.go:127-129`, the `WillSetParam` method sets parameter values without enforcing type checks or validating constraints such as array lengths or value ranges.

As a result, improperly structured or invalid data, such as empty lists or mismatched types, can be written directly to the `KVStore`.

This could result in decoding failures, silent state corruption, or undefined behavior during parameter retrieval or usage elsewhere in the system.

Recommendation

We recommend implementing strict type assertions (e.g., `value.([]string)`) and validation logic (e.g., checking array lengths, reserved values) in `WillSetParam`.

Status: Reported

This audit was performed on an early version of the protocol.

5. Missing struct type validation risks runtime panics during parameter processing

Severity: Minor

In `tm2/pkg/sdk/params/amino_helper.go:15-17`, the code uses reflection to obtain type information for `prm`, but does not verify that `prm` is a struct.

If a non-struct type such as an integer or slice is passed, the function panics with a generic error message. This lack of validation risks obscure runtime crashes during parameter registration, complicating debugging and reducing system robustness.

Recommendation

We recommend adding an explicit kind check before invoking `amino.GetTypeInfo`. If `rvPrm.Kind() != reflect.Struct`, the function should panic with a descriptive error message indicating the expected type.

Status: Reported

This audit was performed on an early version of the protocol.

6. Unrestricted access to coin's lock and unlock proposal creation functions

Severity: Minor

In `examples/gno.land/r/sys/params/unlock.gno`, the `ProposeUnlockTransfer` and `ProposeLockTransfer` functions allow for the submission of proposals to unlock or lock transfers for the `ugnot` coin denomination.

These functions are intended for a one-time use case; however, they are exposed in a way that permits invocation by any realm.

This unrestricted accessibility creates an attack surface where malicious actors could submit excessive or unnecessary proposals, leading to governance process degradation through proposal spam.

Recommendation

We recommend restricting the invocation of `ProposeUnlockTransfer` and `ProposeLockTransfer` to authorized realms only.

Status: Reported

This audit was performed on an early version of the protocol.

7. Missing validation in `willSetKeeperParams`

Severity: Minor

In `gno.land/pkg/sdk/vm/builtins.go:114-128`, the `willSetKeeperParams` function serves as a helper to set parameters of various types.

However, it does not properly handle certain edge cases, particularly when the key contains only a module name without a subkey (e.g., `"vm:"`).

In such cases, the resulting subkey is empty, which could lead to unexpected behavior or runtime errors during parameter handling.

Recommendation

We recommend implementing explicit validation to check for the presence of a non-empty subkey.

Status: Reported

This audit was performed on an early version of the protocol.

8. Incomplete genesis export

Severity: Minor

In `gno.land/pkg/sdk/vm/genesis.go:62-65`, the `ExportGenesis` function of the `VMKeeper` only exports the `Params` field while omitting `RealmParams`.

This results in an incomplete genesis state export, which prevents accurate reconstruction of the full chain state during genesis import.

Such omissions can lead to inconsistencies or failures during chain initialization and state migration.

Recommendation

We recommend updating the `ExportGenesis` function to include `RealmParams` in the exported state.

Status: Reported

This audit was performed on an early version of the protocol.

9. Partial genesis validation

Severity: Minor

In `gno.land/pkg/sdk/vm/genesis.go:29-44`, the `ValidateGenesis` function is responsible for validating the integrity of the genesis state. While it correctly validates the `Params` field, it fails to perform any validation on the `RealmParams`.

This omission can allow invalid or inconsistent `RealmParams` to be included in the genesis file, potentially leading to runtime errors or unexpected behavior during chain initialization.

Recommendation

We recommend extending the `ValidateGenesis` function to include comprehensive validation logic for `RealmParams`.

Status: Reported

This audit was performed on an early version of the protocol.

10. Non-idiomatic bitwise operations

Severity: Informational

In `gno.land/pkg/gnoland/types.go:65`, the code clears a flag using the expression `ga.Attributes &= ^flag`.

Although this operation is functionally correct, it deviates from Go's idiomatic syntax for bitwise clearing, which uses the `&^=` operator.

This non-standard usage may confuse developers during code review or maintenance, as it obscures intent and diverges from established Go conventions.

Recommendation

We recommend replacing `ga.Attributes &= ^flag` with the idiomatic `ga.Attributes &^= flag` to align with Go conventions, improving readability and reducing cognitive overhead for future contributors.

Status: Reported

This audit was performed on an early version of the protocol.

11. Missing explicit realm path check reduces code clarity

Severity: Informational

In `gnovm/stdlibs/std/params.go:61`, the code performs character validation on the key but does not explicitly prevent the `rlmPath` (realm path) from being set to the reserved value `"p"`.

Although `"p"` may not typically occur in practice, the lack of a dedicated check introduces ambiguity and creates the potential for accidental overwrites of internal VM state reserved for `"p"`. This omission reduces clarity and maintainability, increasing the risk of subtle bugs in future development.

Recommendation

We recommend adding an explicit check such as if `rlmPath == "p" { panic(...) }` to clarify intent and guard against hypothetical edge cases..

Status: Reported

This audit was performed on an early version of the protocol.

12. Missing parameter getters limit runtime validation and debugging

Severity: Informational

In `gnovm/stdlibs/sys/params/params.gno`, the code defines `setSysParam*` functions (e.g., `setSysParamInt64`) for writing system parameters but does not provide corresponding getter functions (e.g., `getSysParamInt64`).

This asymmetry prevents modules from programmatically accessing configured system parameters during execution, limiting the ability to validate parameter values, debug state-dependent behavior, or verify consistency between configured and active settings.

Recommendation

We recommend implementing getter functions (e.g., `getSysParamInt64`, `getSysParamUint64`) for each parameter type to enable safe read access.

Status: Reported

This audit was performed on an early version of the protocol.

13. Usage of panics instead of structured error handling

Severity: Informational

In `tm2/pkg/sdk/bank/msgs.go:172-179`, the `ValidateBasic` function calls `Add`, which panics in case of any error. While a panic in `ValidateBasic` would not crash the node due to the SDK's panic recovery mechanisms, relying on panic recovery is not a recommended practice:

- **Missed error handling**: panics are not subject to type-checking, so it is easier to overlook error-handling mechanisms.
- **Unrecoverable failures**: if a panic signals a recoverable error and is not properly handled, the program may terminate unnecessarily.
- **Reduced readability and debugging complexity**: codebases that rely on panics are harder to follow and debug, increasing maintenance overhead.
- **Performance overhead**: unwinding the stack decreases program performance.

Consequently, relying on panics as the primary error-handling mechanism increases the difficulty of ensuring codebase safety and maintainability.

Recommendation

We recommend returning errors with meaningful error messages rather than using panics. This approach improves code maintainability, debugging efficiency, and overall developer experience.

Status: Reported

This audit was performed on an early version of the protocol.

14. Redundant validations

Severity: Informational

In `tm2/pkg/sdk/bank/msgs.go:115-117,143-145`, the code attempts to validate that input and output addresses are not zero-length, returning `ErrInvalidAddress` if an address length equals zero.

However, the `Address` type is defined as a fixed-size array, specifically `[AddressSize]byte`, with `AddressSize` currently set to `20`.

As a result, address values are always `20` bytes long, and the length check always passes. This renders the validation ineffective, and `ErrInvalidAddress` is never triggered in practice.

Recommendation

We recommend removing redundant validations.

Status: Reported

This audit was performed on an early version of the protocol.

15. Missing check for distinct sender and recipient allows redundant transfers

Severity: Informational

In `tm2/pkg/sdk/bank/msgs.go:34-48`, the `ValidateBasic` function for the `MsgSend` message performs basic validations but does not check whether `FromAddress` and `ToAddress` are distinct.

This omission allows transfers where the sender and recipient are the same, resulting in semantically meaningless transactions that may lead to confusion or unnecessary state transitions.

Recommendation

We recommend adding a validation to ensure that `FromAddress` and `ToAddress` are not equal, thereby enforcing that all transfers involve distinct parties.

Status: Reported

This audit was performed on an early version of the protocol.

16. Outstanding `TODO` and `XXX` comments

Severity: Informational

The codebase contains numerous `TODO` and `XXX` comments. This approach to documenting issues and outstanding work has a few disadvantages:

- `TODO` comments often become stale compared to properly tracked issues, leading to a lack of progress toward resolution.
- Information in comments must be maintained and updated. Otherwise, it may be misleading for future developers or reviewers.
- These comments highlight weak points in the codebase, which could be exploited by malicious actors since the code is publicly accessible.

A non-exhaustive list of discovered `TODO` comments includes:

- `tm2/pkg/sdk/params/types.go:48` highlights missing validation in `ValidateBasic`
- `gno.land/pkg/sdk/vm/genesis.go:40` and `gno.land/pkg/gnoland/genesis.go:123` can be addressed using already existing functions in the codebase
- `gno.land/pkg/gnoland/types.go:24` is outdated since the flags are already prefixed in the current version of the codebase
- `gno.land/pkg/gnoland/types.go:29,32` are not informative enough

The total number of such comments exceeds `200` across the codebase.

Recommendation

We recommend addressing the most pressing `TODO` and `XXX` comments while moving the remaining ones into a structured issue-tracking system.

Status: Reported

This audit was performed on an early version of the protocol.

17. Misleading comments

Severity: Informational

In `tm2/pkg/sdk/bank/msgs.go:22-25`, the `NewMsgSend` function constructs a `MsgSend` structure for single-input, single-output transfers.

However, the accompanying in-line documentation incorrectly states that it creates a *"multi-in, multi-out send msg"*.

This misleading comment likely results from a copy-paste error from the `NewMsgMultiSend` function, which does support multiple inputs and outputs.

Recommendation

We recommend correcting in-line docs.

Status: Reported

This audit was performed on an early version of the protocol.